Pontificia Universidad Católica de Chile
Escuela de Ingeniería
Departamento de Ciencia de la Computación

Clase 24: Patrones Estructurales (parte 3)

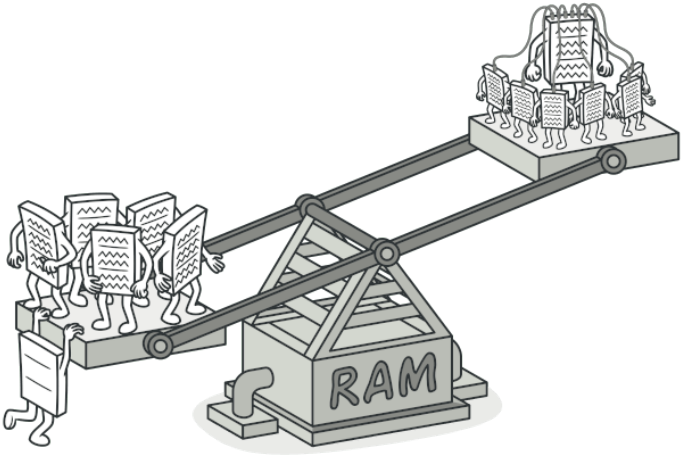
Francisco Gazitúa Requena (fco_gazitua_requena@uc.cl)

IIC2113 Diseño Detallado de Software

30 de Octubre, 2025

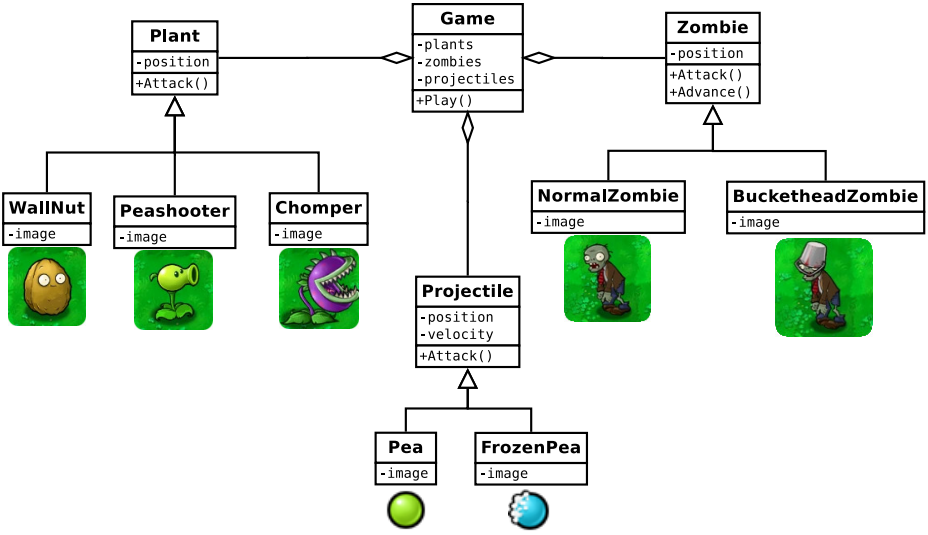
Flyweight

Flyweight

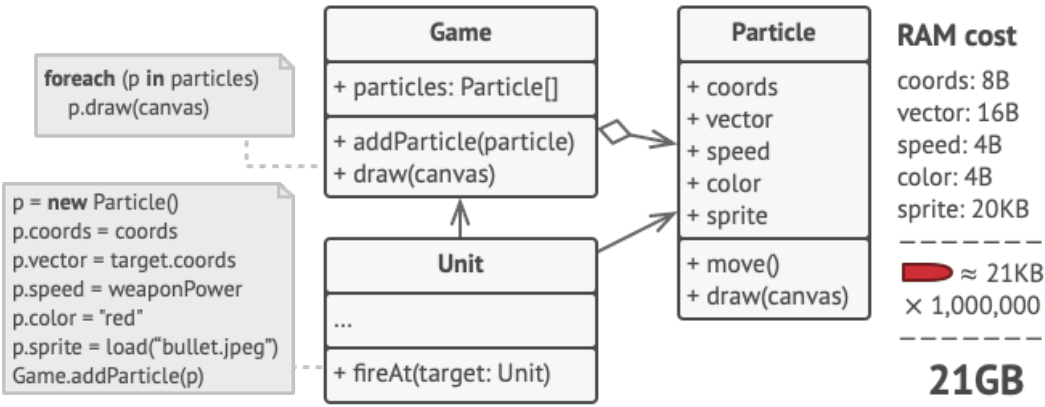


Flyweight

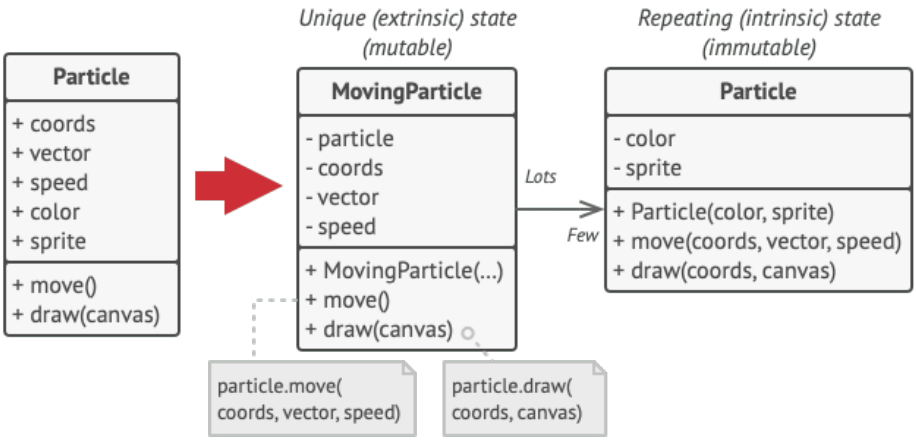
Flyweight


Plant
-position
+Attack()
Game
-plants
-zombies
-projectiles
+Play()
Zombie
-position
+Attack()
+Advance()
WallNut
-image
Peashooter
-image
Chomper
-image
NormalZombie
-image
BucketheadZombie
-image
Projectile
-position
-velocity
+Attack()
Pea
-image
FrozenPea
-image

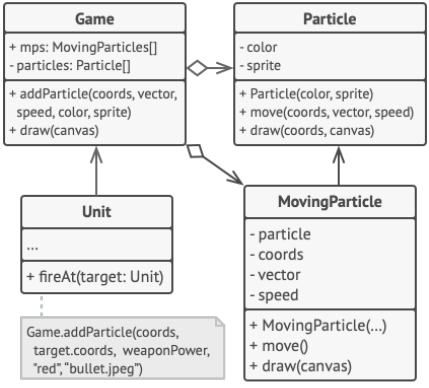
Flyweight


foreach (p in particles)
p.draw(canvas)
p = new Particle()
p.coords = coords
p.vector = target.coords
p.speed = weaponPower
p.color = "red"
p.sprite = load("bullet.jpeg")
Game.addParticle(p)
Game
+ particles: Particle[]
+ addParticle(particle)
+ draw(canvas)
Unit
...
+ fireAt(target: Unit)
Particle
+ coords
+ vector
+ speed
+ color
+ sprite
+ move()
+ draw(canvas)
RAM cost
coords: 8B
vector: 16B
speed: 4B
color: 4B
sprite: 20KB
≈ 21KB
× 1,000,000
21GB

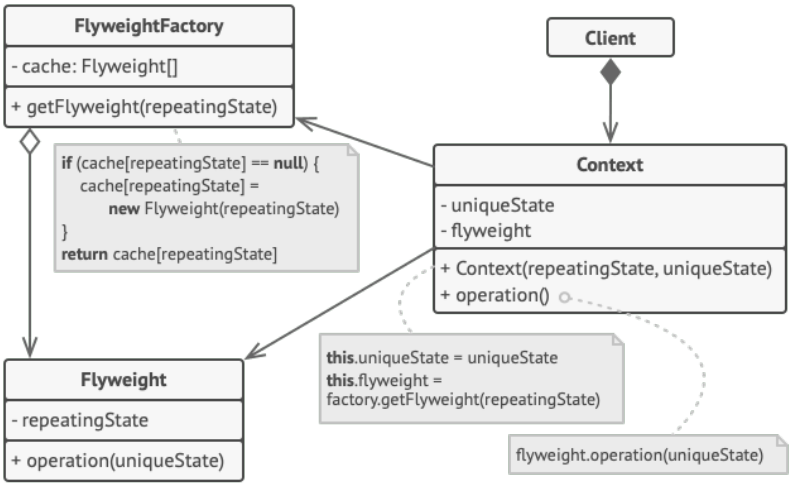
Flyweight


Particle
+ coords
+ vector
+ speed
+ color
+ sprite
+ move()
+ draw(canvas)
Unique (extrinsic) state
(mutable)
MovingParticle
- particle
- coords
- vector
- speed
+ MovingParticle(...)
+ move()
+ draw(canvas)
Lots
Few
Repeating (intrinsic) state
(immutable)
Particle
- color
- sprite
+ Particle(color, sprite)
+ move(coords, vector, speed)
+ draw(coords, canvas)
particle.move(
coords, vector, speed)
particle.draw(
coords, canvas)

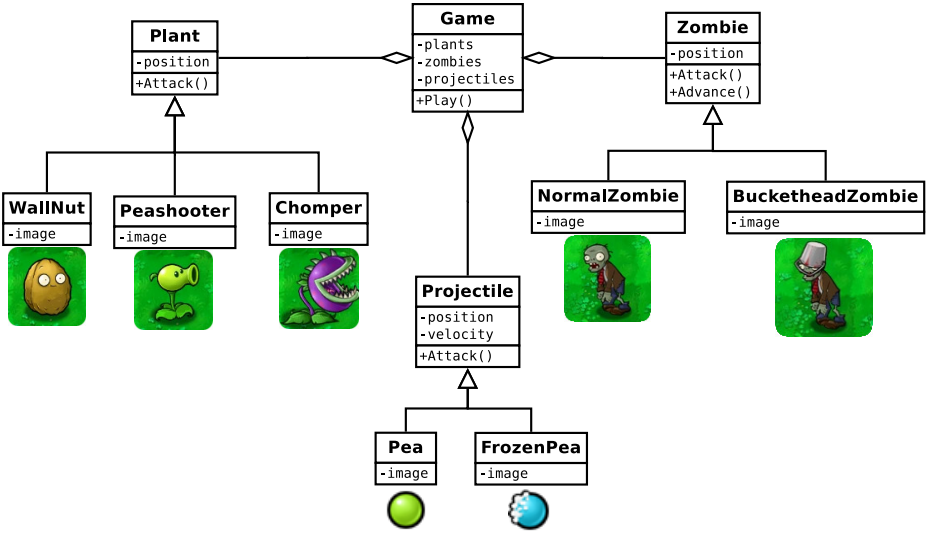
Flyweight


Game
+ mps: MovingParticles[]
- particles: Particle[]
+ addParticle(coords, vector, speed, color, sprite)
+ draw(canvas)
Particle
- color
- sprite
+ Particle(color, sprite)
+ move(coords, vector, speed)
+ draw(coords, canvas)
Unit
...
+ fireAt(target: Unit)
MovingParticle
- particle
- coords
- vector
- speed
+ MovingParticle(...)
+ move()
+ draw(canvas)
Game.addParticle(coords, target.coords, weaponPower, "red","bullet.jpeg")
RAM cost
color: 4B
sprite: 20KB
≈ 21KB
coords: 8B
vector: 16B
speed: 4B
particle: 4B
≈ 32B
× 1
× 1,000,000
32MB

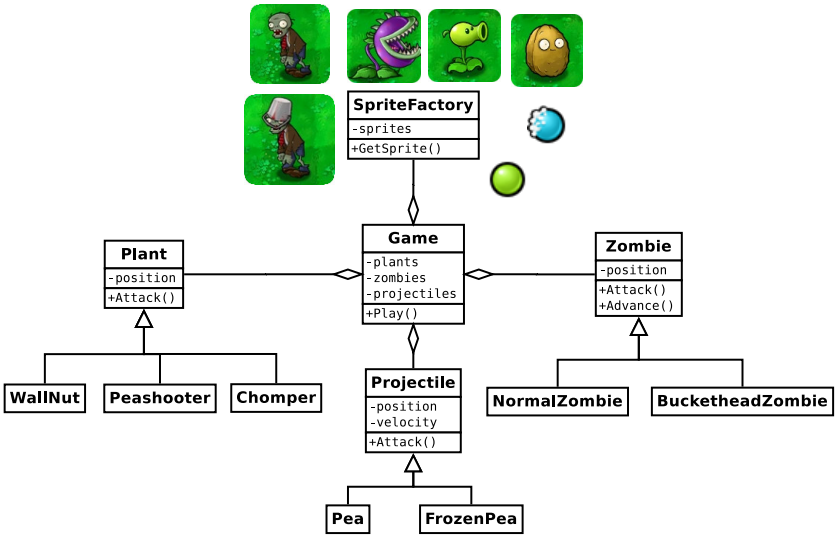
Flyweight


FlyweightFactory
- cache: Flyweight[]
+ getFlyweight(repeatingState)
if (cache[repeatingState] == null) {
cache[repeatingState] =
new Flyweight(repeatingState)
}
return cache[repeatingState]
Client
Context
- uniqueState
- flyweight
+ Context(repeatingState, uniqueState)
+ operation()
this.uniqueState = uniqueState
this.flyweight =
factory.getFlyweight(repeatingState)
flyweight.operation(uniqueState)
Flyweight
- repeatingState
+ operation(uniqueState)

Flyweight

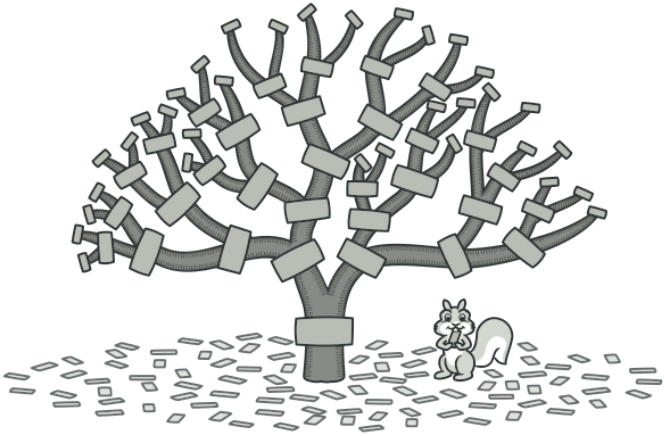

Plant
-position
+Attack()
Game
-plants
-zombies
-projectiles
+Play()
Zombie
-position
+Attack()
+Advance()
WallNut
-image
Peashooter
-image
Chomper
-image
NormalZombie
-image
BucketheadZombie
-image
Projectile
-position
-velocity
+Attack()
Pea
-image
FrozenPea
-image

Flyweight

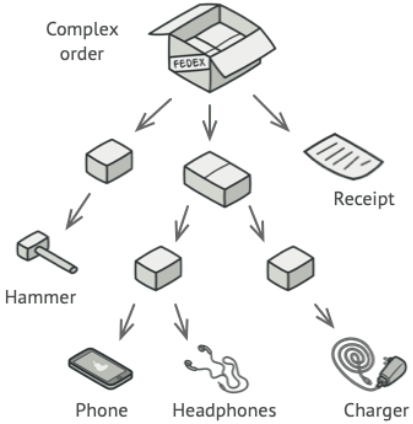

SpriteFactory
-sprites
+GetSprite()
Game
-plants
-zombies
-projectiles
+Play()
Plant
-position
+Attack()
Zombie
-position
+Attack()
+Advance()
WallNut
Peashooter
Chomper
Projectile
-position
-velocity
+Attack()
NormalZombie
BucketheadZombie
Pea
FrozenPea

Composite

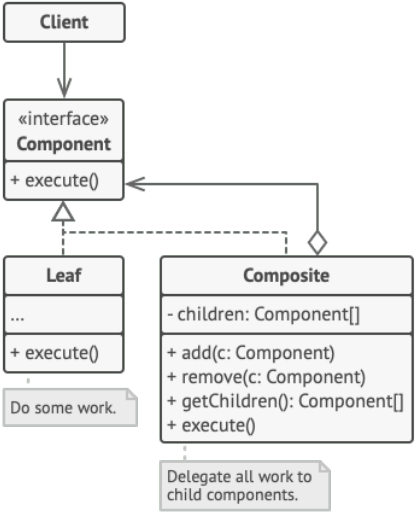
Composite



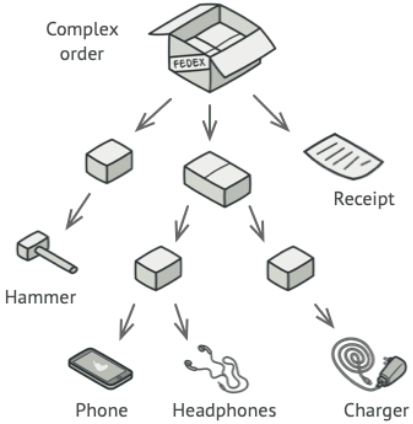
Composite


Complex order
FEDEX
Receipt
Hammer
Phone
Headphones
Charger

Composite


Client
«interface»
Component
+ execute()
Leaf
...
+ execute()
Do some work.
Composite
- children: Component[]
+ add(c: Component)
+ remove(c: Component)
+ getChildren(): Component[]
+ execute()
Delegate all work to
child components.

Composite


Complex order
FEDEX
Receipt
Hammer
Phone
Headphones
Charger

Composite

Comencemos con la interfaz de un componente:

```
1 public interface Component {
2     void ShowItems();
3     int GetTotalPrice();
4 }
```

Composite

Ahora creemos un item cualquiera (que es una hoja del árbol):

```
public class Item : Component {
    private readonly string _name;
    private readonly int _price;

    public Item(string name, int price) {
        _name = name;
        _price = price;
    }

    public void ShowItems()
        => Console.WriteLine($"{_name} => ${_price}USD.");

    public int GetTotalPrice()
        => _price;
}
```

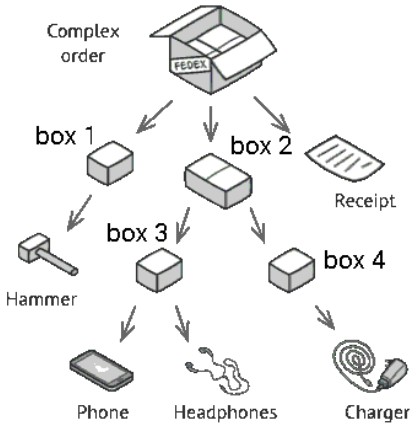
Composite

Las cajas deben permitir agregar items, mostrar sus items y obtener su precio:

```
public class Box : Component {
    private List<Component> components = new List<Component>();
    public void Add(Component component)
        => components.Add(component);

    public void ShowItems() {
        foreach (var component in components)
            component.ShowItems();
    }

    public int GetTotalPrice() {
        int total = 0;
        foreach (var component in components)
            total += component.GetTotalPrice();
        return total;
    }
}
```

Composite

Luego podemos crear cajas e items:



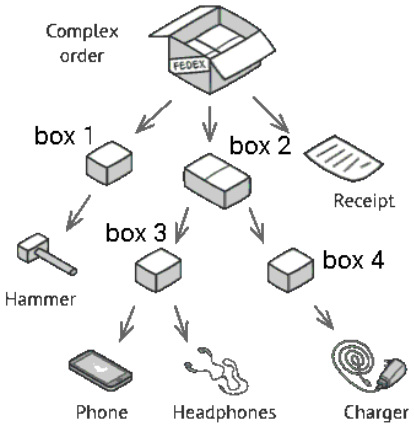
Composite

Luego podemos crear cajas e items:

```
Box order = new Box();
Box box1 = new Box();
Box box2 = new Box();
Box box3 = new Box();
Box box4 = new Box();
Component receipt = new Item("receipt", 1);
Component hammer = new Item("hammer", 9);
Component phone = new Item("phone", 829);
Component headphones = new Item("headphones", 30);
Component charger = new Item("charger", 14);
```

Composite

Poner los items en cajas:



Composite

Poner los items en cajas:

```
12 order.Add(box1);
13 order.Add(box2);
14 order.Add(receipt);
15 box1.Add(hammer);
16 box2.Add(box3);
17 box2.Add(box4);
18 box3.Add(phone);
19 box3.Add(headphones);
20 box4.Add(charger);
```

Composite

Poner los items en cajas:

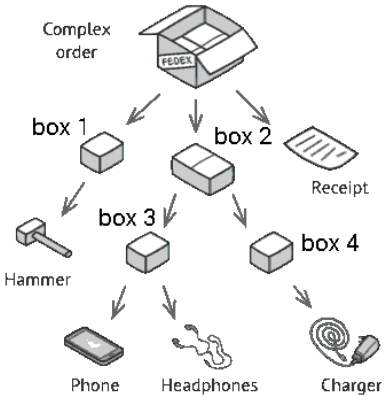
```
12 order.Add(box1);
13 order.Add(box2);
14 order.Add(receipt);
15 box1.Add(hammer);
16 box2.Add(box3);
17 box2.Add(box4);
18 box3.Add(phone);
19 box3.Add(headphones);
20 box4.Add(charger);
```

... y obtener información general de la orden:

```
22 Console.WriteLine("-------------------------");
23 order.ShowItems();
24 Console.WriteLine("-------------------------");
25 Console.WriteLine($"Total => ${order.GetTotalPrice()}USD");
```

Composite

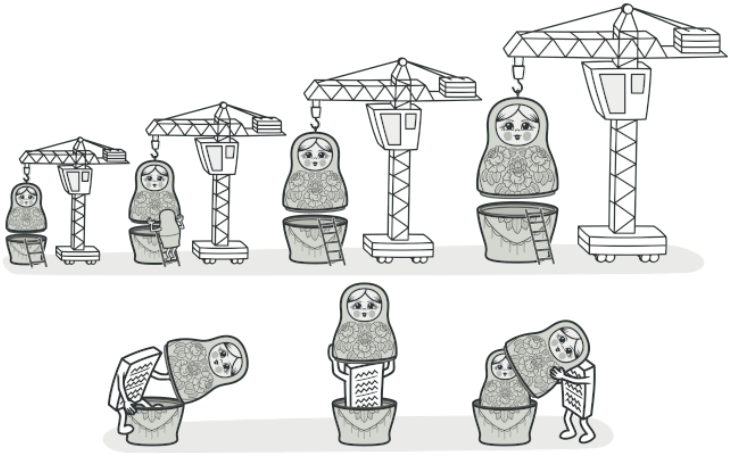
... y obtener información general de la orden:



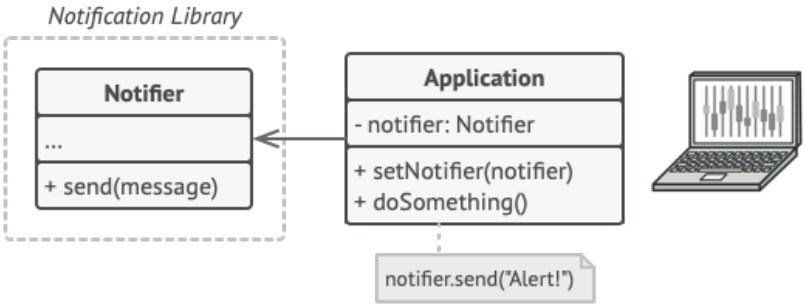

```
1 -------------------------
2 hammer => $9USD.
3 phone => $829USD.
4 headphones => $30USD.
5 charger => $14USD.
6 receipt => $1USD.
7 -------------------------
8 Total => $883USD
```

Decorator

Decorator

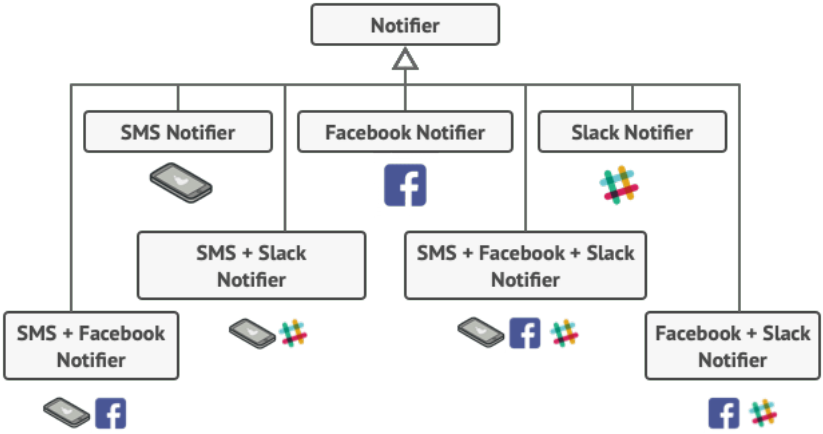


Decorator

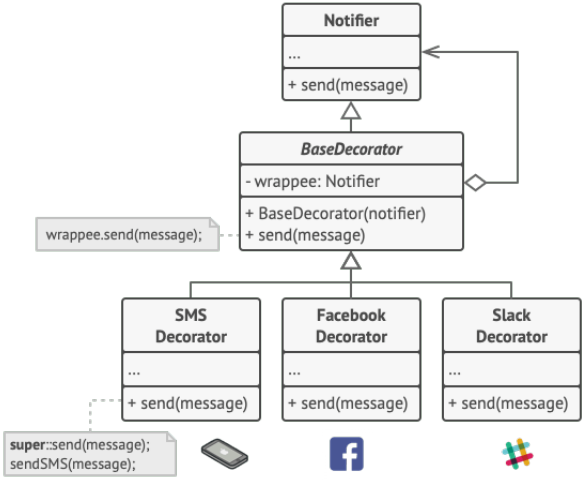

Notification Library
Notifier
...
+ send(message)
Application
- notifier: Notifier
+ setNotifier(notifier)
+ doSomething()
notifier.send("Alert!")

Decorator

Decorator

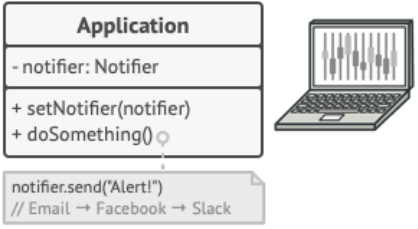

Notifier
SMS Notifier
Facebook Notifier
Slack Notifier
SMS + Slack Notifier
SMS + Facebook + Slack Notifier
SMS + Facebook Notifier
Facebook + Slack Notifier

Decorator

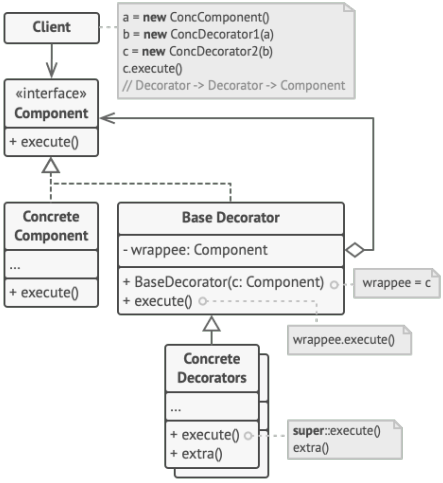

Notifier
...
+ send(message)
BaseDecorator
- wrappee: Notifier
+ BaseDecorator(notifier)
+ send(message)
wrappee.send(message);
SMS Decorator
...
+ send(message)
Facebook Decorator
...
+ send(message)
Slack Decorator
...
+ send(message)
super::send(message);
sendSMS(message);

Decorator

```
stack = new Notifier()
if (facebookEnabled)
    stack = new FacebookDecorator(stack)
if (slackEnabled)
    stack = new SlackDecorator(stack)

app.setNotifier(stack)
```



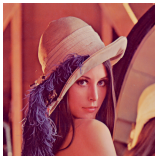
Decorator


Client
a = new ConcComponent()
b = new ConcDecorator1(a)
c = new ConcDecorator2(b)
c.execute()
// Decorator -> Decorator -> Component
«interface»
Component
+ execute()
Concrete
Component
...
+ execute()
Base Decorator
- wrappee: Component
+ BaseDecorator(c: Component)
+ execute()
wrappee = c
wrappee.execute()
Concrete
Decorators
...
+ execute()
+ extra()
super::execute()
extra()

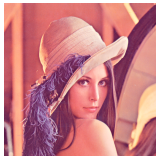
Actividad

Photoshop

Veamos un pequeño programa que simula el Photoshop:



(a) Oscurecer



(b) Aclarar

(c) Rotar

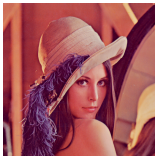
(d) Reflejar

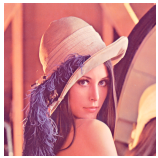
(e) Difuminar

(f) Bordes

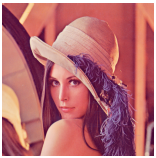
Photoshop

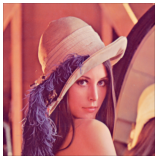
Veamos un pequeño programa que simula el Photoshop:


(a) Oscurecer


(b) Aclarar


(c) Rotar


(d) Reflejar

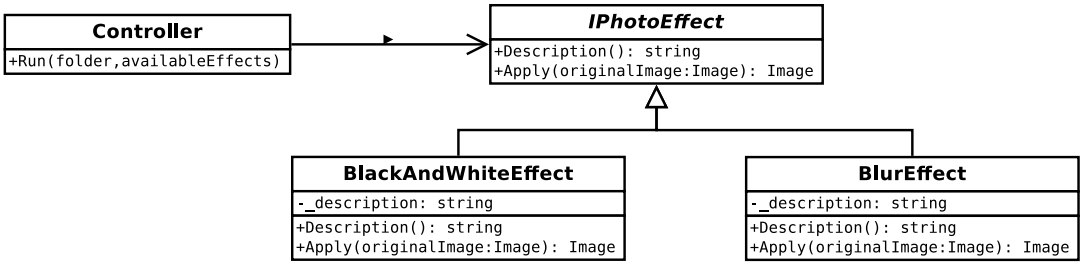

(e) Difuminar


(f) Bordes

Desafío: Sin cambiar el código del `controller` (puedes cambiar el main), permite que el usuario pueda aplicar los efectos BlackAndWhite y Blur al mismo tiempo.

Photoshop

Veamos un pequeño programa que simula el Photoshop:

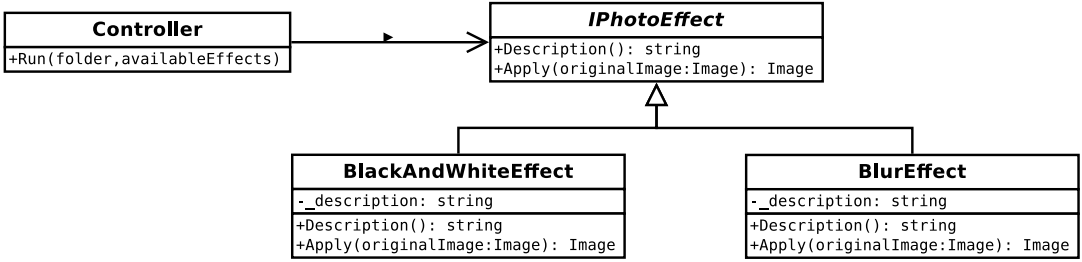


Desafío: Sin cambiar el código del `controller` (puedes cambiar el main), permite que el usuario pueda aplicar los efectos BlackAndWhite y Blur al mismo tiempo.

Photoshop

```
¡Bienvenido!
En este minuto existen las siguientes imágenes en tu directorio:
/home/rodrigo/Desktop/IIC2113/Clases/clase 12/actividad/
    MyPhotoshop_proxy/MyPhotoshop/bin/Debug/net6.0/imgs

0- imgs/Lenna.png
Ingresa una opción válida:
0

¿Qué efecto deseas usar en la imagen?
0- Pasar a blanco y negro.
1- Difuminar.
2- Aumentar brillo.
3- Pasar a blanco y negro. Difuminar.
Ingresa una opción válida:
3

Aplicando el efecto... listo!
```

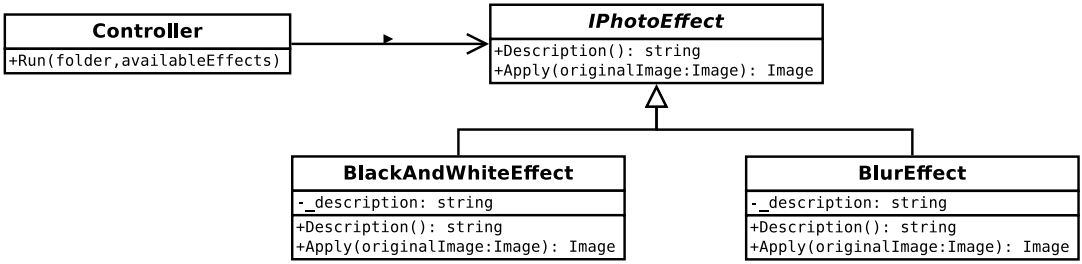
Photoshop



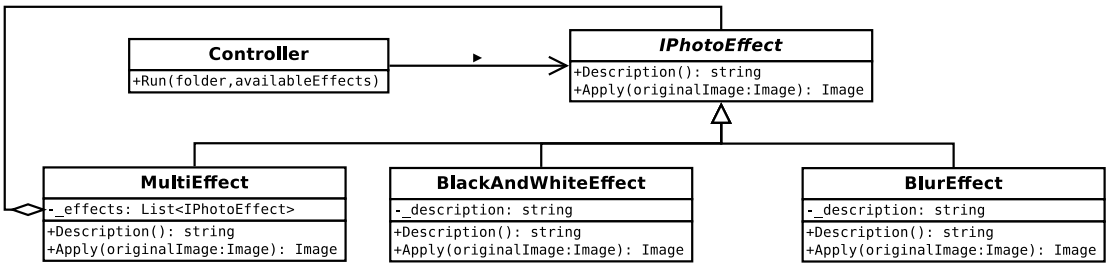
Además debe ser fácil agregar nuevas combinaciones de efectos a futuro.

Solución

Solución


Controller
+Run(folder,availableEffects)
IPhotoEffect
+Description(): string
+Apply(originalImage:Image): Image
BlackAndWhiteEffect
-_description: string
+Description(): string
+Apply(originalImage:Image): Image
BlurEffect
-_description: string
+Description(): string
+Apply(originalImage:Image): Image

Solución


Controller
+Run(folder,availableEffects)
IPhotoEffect
+Description(): string
+Apply(originalImage:Image): Image
MultiEffect
-_effects: List<IPhotoEffect>
+Description(): string
+Apply(originalImage:Image): Image
BlackAndWhiteEffect
-_description: string
+Description(): string
+Apply(originalImage:Image): Image
BlurEffect
-_description: string
+Description(): string
+Apply(originalImage:Image): Image

Solución

```
public class MultiEffect:IPhotoEffect{
  private List<IPhotoEffect> _effects;
  public MultiEffect(List<IPhotoEffect> effects)
    => _effects = effects;
  public string Description {
    get{
      string[] descriptions = new string[_effects.Count];
      for(int i = 0; i < _effects.Count; i++)
        descriptions[i] = _effects[i].Description;
      return String.Join(' ', descriptions);
    }
  }
  public Image<Rgb24> Apply(Image<Rgb24> originalImage) {
    Image<Rgb24> newImage = originalImage;
    foreach (var effect in _effects)
      newImage = effect.Apply(newImage);
    return newImage;
  }
}
```

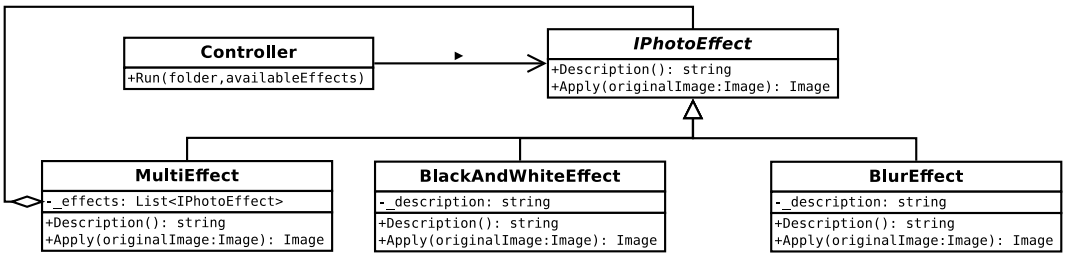
Solución

En el `main` podemos crear el `MultiEffects` que queramos (sin crear nuevas clases):

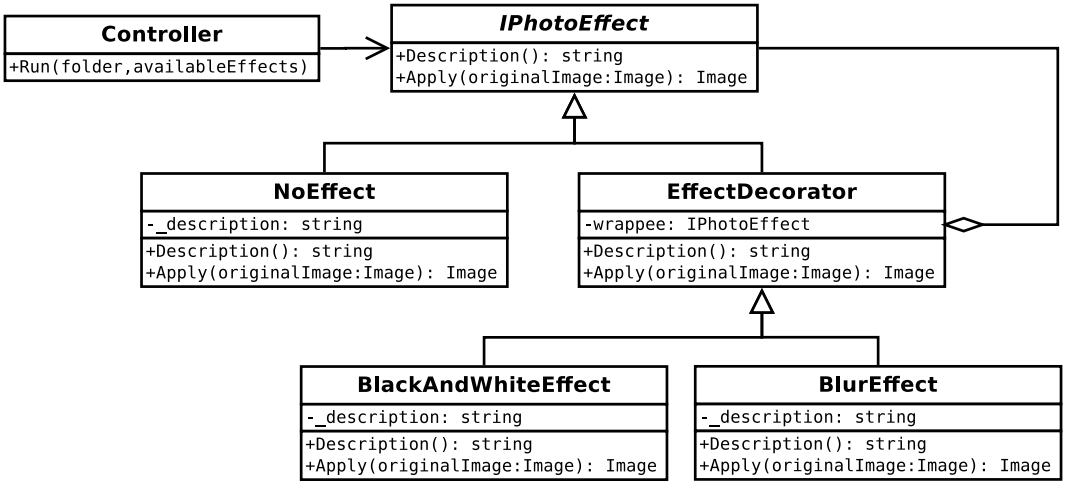
```
IPhotoEffect blackAndWhite = new BlackAndWhiteEffect();
IPhotoEffect blur = new BlurEffect();
IPhotoEffect brightness = new IncreaseBrightnessEffect();

IPhotoEffect blurAndBright = new MultiEffect(
  new List<IPhotoEffect>() { blur, brightness });

IPhotoEffect[] availableEffects = new IPhotoEffect[]
    { blackAndWhite, blur, brightness, blurAndBright };
string folder = "imgs";
Controller.Run(folder, availableEffects);
```

Veamos una solución alternativa

Decorator


Controller
+Run(folder,availableEffects)
IPhotoEffect
+Description(): string
+Apply(originalImage:Image): Image
MultiEffect
-_effects: List<IPhotoEffect>
+Description(): string
+Apply(originalImage:Image): Image
BlackAndWhiteEffect
-_description: string
+Description(): string
+Apply(originalImage:Image): Image
BlurEffect
-_description: string
+Description(): string
+Apply(originalImage:Image): Image

Decorator


Controller
+Run(folder,availableEffects)
IPhotoEffect
+Description(): string
+Apply(originalImage:Image): Image
NoEffect
-_description: string
+Description(): string
+Apply(originalImage:Image): Image
EffectDecorator
-wrappee: IPhotoEffect
+Description(): string
+Apply(originalImage:Image): Image
BlackAndWhiteEffect
-_description: string
+Description(): string
+Apply(originalImage:Image): Image
BlurEffect
-_description: string
+Description(): string
+Apply(originalImage:Image): Image

Decorator

```
2 public class NoEffect:IPhotoEffect {
3     public string Description => "";
4
5     public Image<Rgb24> Apply(Image<Rgb24> originalImage)
6         => originalImage;
7 }
```

Decorator

```
2  public abstract class EffectDecorator : IPhotoEffect {
3      private IPhotoEffect wrappee;
4      public EffectDecorator(IPhotoEffect wrappee)
5          => this.wrappee = wrappee;
6
7      public abstract string EffectDescription { get; }
8      public string Description {
9          get {
10             string description = wrappee.Description;
11             description += EffectDescription + " ";
12             return description;
13         }
14     }
15
16     public virtual Image<Rgb24> Apply(Image<Rgb24> originalImage)
17         => wrappee.Apply(originalImage);
18 }
```

Decorator

```
2  public class BlurEffect:EffectDecorator {
3      private Image<Rgb24> _originalImage;
4      private int _width;
5      private int _height;
6
7      public BlurEffect(IPhotoEffect wrappee) : base(wrappee) { }
8
9      public override string EffectDescription => "Difuminar.";
10
11     public override Image<Rgb24> Apply(Image<Rgb24> originalImage) {
12         originalImage = base.Apply(originalImage);
13         return ApplyBlurEffect(originalImage);
14     }
15     /* ... ETC ... */
16 }
```

Decorator

```
IPhotoEffect noEffect = new NoEffect();

IPhotoEffect blackAndWhite = new BlackAndWhiteEffect(noEffect);
IPhotoEffect blur = new BlurEffect(noEffect);
IPhotoEffect brightness = new IncreaseBrightnessEffect(noEffect);

IPhotoEffect blurAndBright = new IncreaseBrightnessEffect(blur);

IPhotoEffect[] availableEffects = new IPhotoEffect[]
    { blackAndWhite, blur, brightness, blurAndBright };
string folder = "imgs";
Controller.Run(folder, availableEffects);
```

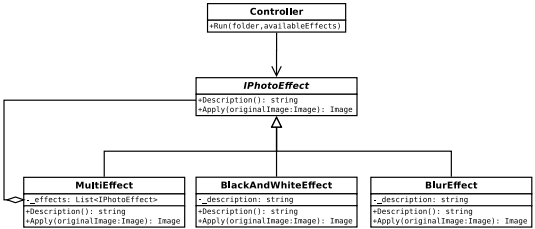
... y así se crean efectos en el `main`.

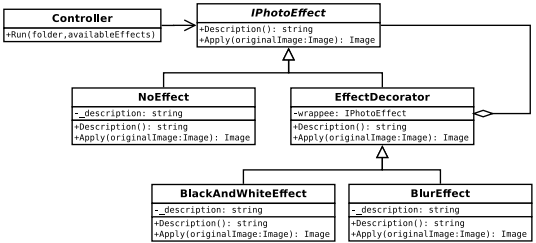
Decorator

```
¡Bienvenido!
En este minuto existen las siguientes imágenes en tu directorio:
/home/rodrigo/Desktop/IIC2113/Clases/clase 12/actividad/
    MyPhotoshop_decorator/MyPhotoshop/bin/Debug/net6.0/imgs

0- imgs/Lenna.png
Ingresa una opción válida:
0

¿Qué efecto deseas usar en la imagen?
0- Pasar a blanco y negro.
1- Difuminar.
2- Aumentar brillo.
3- Difuminar. Aumentar brillo.
Ingresa una opción válida:
3
```

Decorator

¿Qué solución es mejor? ¿son ambos modelos equivalentes?





Decorator

```
IPhotoEffect blackAndWhite = new BlackAndWhiteEffect();
IPhotoEffect blur = new BlurEffect();
IPhotoEffect brightness = new IncreaseBrightnessEffect();

IPhotoEffect blurAndBright = new MultiEffect(
  new List<IPhotoEffect>() { blur, brightness });

IPhotoEffect[] availableEffects = new IPhotoEffect[]
    { blackAndWhite, blur, brightness, blurAndBright };
string folder = "imgs";
Controller.Run(folder, availableEffects);
```

Decorator

```
IPhotoEffect noEffect = new NoEffect();

IPhotoEffect blackAndWhite = new BlackAndWhiteEffect(noEffect);
IPhotoEffect blur = new BlurEffect(noEffect);
IPhotoEffect brightness = new IncreaseBrightnessEffect(noEffect);

IPhotoEffect blurAndBright = new IncreaseBrightnessEffect(blur);

IPhotoEffect[] availableEffects = new IPhotoEffect[]
    { blackAndWhite, blur, brightness, blurAndBright };
string folder = "imgs";
Controller.Run(folder, availableEffects);
```